Universidad Ecotec

ACTIVIDAD INTEGRADORA 2

TASMANAGER

Asignatura:

Programacion de Sistemas Web

Estudiante:

Kristhel Annabelle Carbo Fabara

Docente:

Ing. Gregorio manuel cruz gonzabay

Objetivo del proyecto

Desarrollar una aplicación de gestión de tareas utilizando JavaScript Vanilla (ES6+), manipulación del DOM, consumo de una API REST y almacenamiento local en el navegador, con posibilidad de utilización sin conexión después de guardar los recursos necesarios.

Repositorio público

https://github.com/krcarbo-art/ACTIVIDAD-INTEGRADORA-2

1. **Funcionalidades implementadas**

Gestión de tareas (CRUD). Permite crear, editar, eliminar y marcar tareas como completadas. Incluye validación de títulos vacíos desde JavaScript.

Búsqueda y filtros. Busca por título mientras se escribe y permite mostrar todas las tareas, las pendientes o las completadas.

Ordenamiento y prioridades. Clasifica las tareas por fecha de creación o por prioridad alta, media y baja mediante el método sort().

Panel de métricas. Actualiza los contadores de tareas totales, pendientes y completadas cuando cambian los datos.

Persistencia y consulta API. Guarda tareas en localStorage usando JSON.stringify() y las recupera con JSON.parse(). El módulo API consulta JSONPlaceholder con fetch y async/await y presenta errores de conexión.

Uso sin conexión. Se incorporó un Service Worker para almacenar los archivos estáticos del sitio. Se comprobó la carga de la interfaz en modo Offline; las modificaciones sin conexión deben verificarse antes de la entrega.

2. **Investigación técnica: Arquitectura Modular ES6**

Se eligió la opción A de la asignación: dividir la aplicación en módulos JavaScript nativos mediante import y export. Esta técnica separa responsabilidades, facilita las correcciones y evita concentrar toda la lógica en un solo archivo.

Archivo	Responsabilidad
script.js	Coordina eventos, CRUD, filtros, ordenamiento y estadísticas.
dom.js	Crea los elementos y controles de cada tarea en el DOM.
storage.js	Guarda y recupera los datos del navegador.
api.js	Solicita tareas a JSONPlaceholder y maneja errores.
sw.js	Almacena archivos estáticos en caché para abrir el sitio sin red.

3. Evidencia de persistencia local

Captura de Chrome DevTools (Application > Local Storage), en la que se observa la clave «tareas» y un registro almacenado con su título, estado y prioridad.

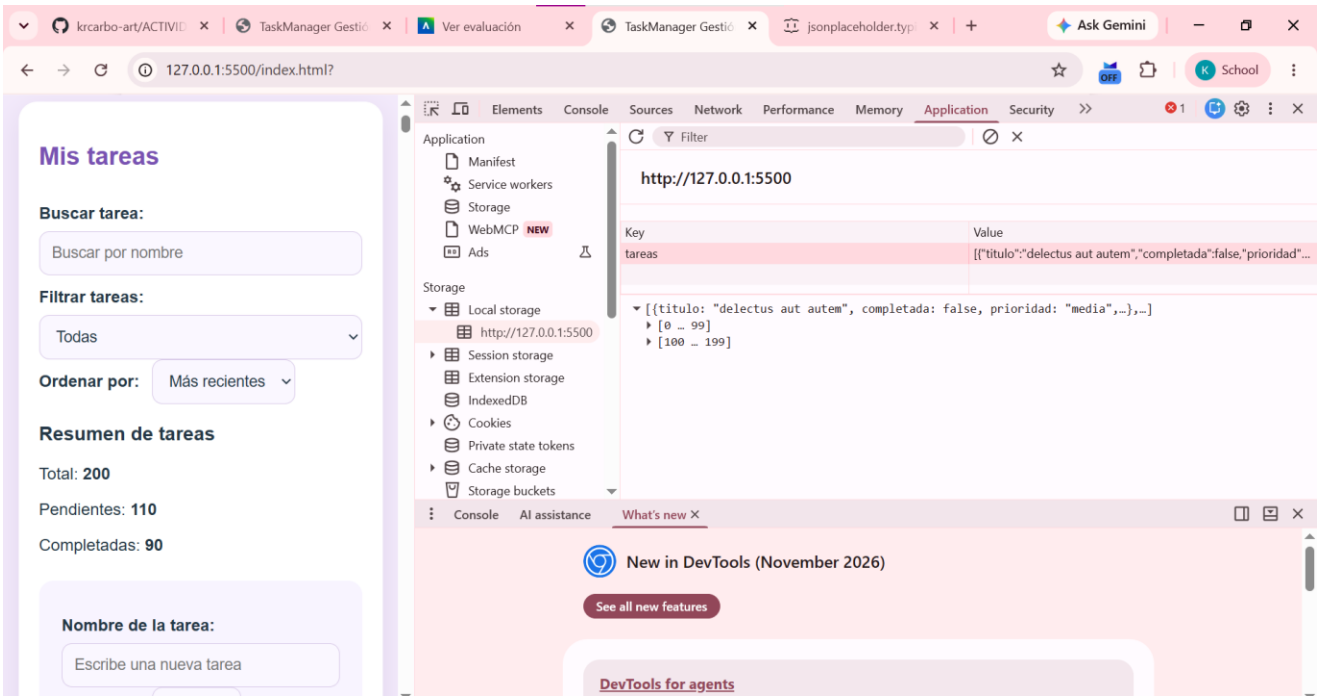
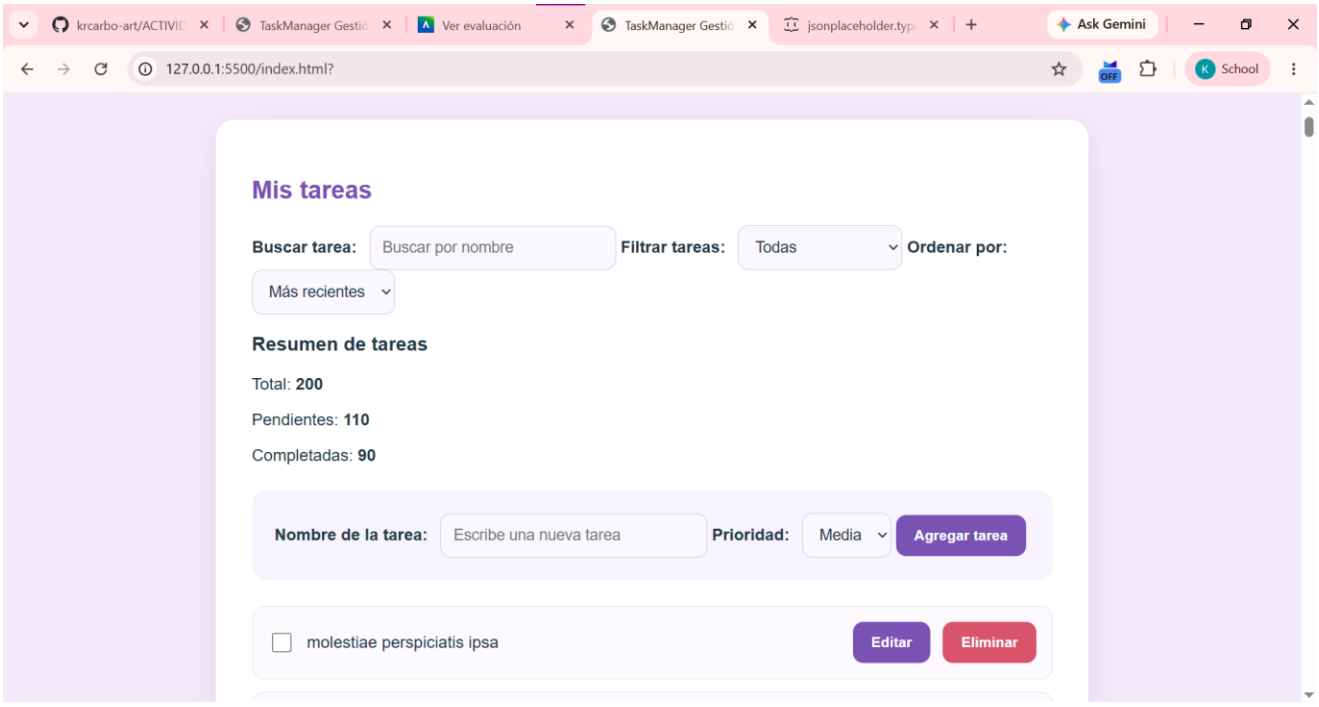
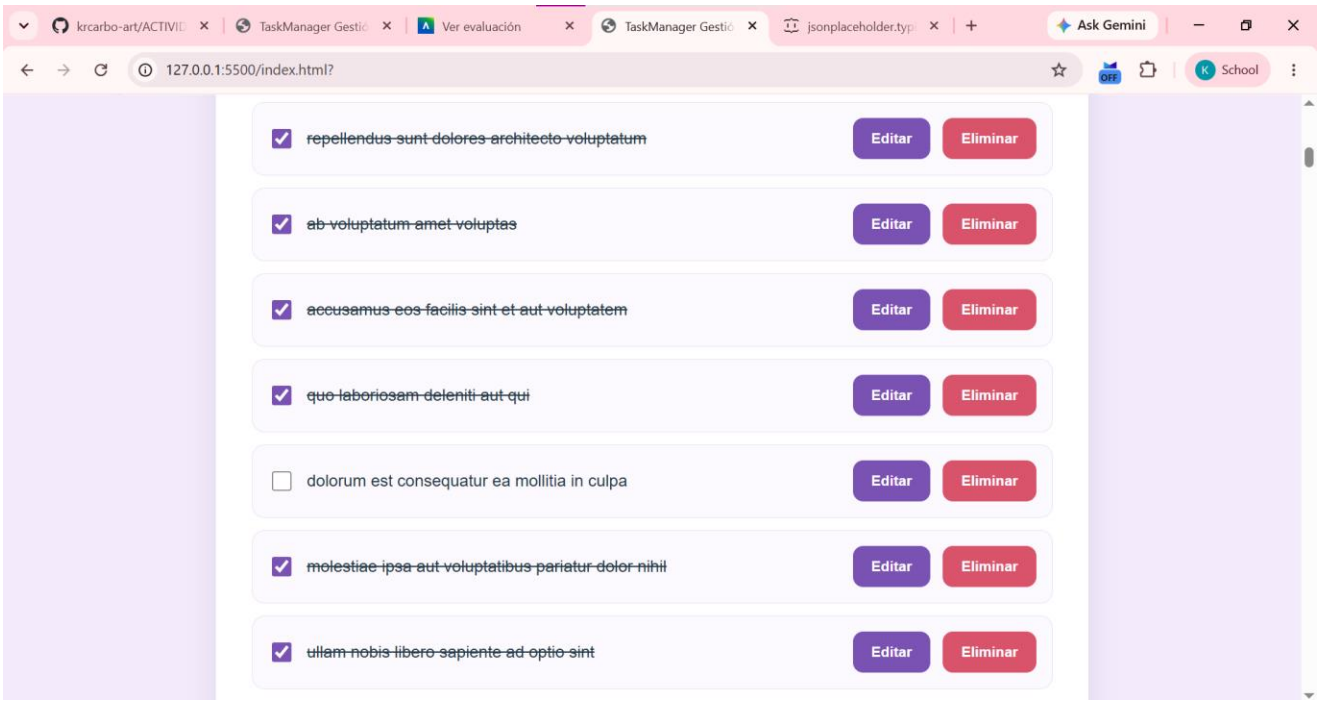


Figura 1. Evidencia de persistencia de datos mediante localStorage.





4. Verificación y entrega

- Repositorio público con historial visible de 35 commits (revisar que al menos cinco sean significativos).
- Archivo AI_DECLARATION.md: verificar su presencia en la raíz del repositorio y que incluya herramienta, dos prompts y ajustes manuales.

- La captura de localStorage se incluye como evidencia. Para completar la verificación, probar creación y persistencia de tareas en modo Offline.

Conclusión

TaskManager integra técnicas esenciales de JavaScript ES6+, manejo del DOM, almacenamiento persistente y consumo de servicios externos. La estructura modular permite mantener organizados los componentes de interfaz, almacenamiento y API. La captura incorporada evidencia el almacenamiento local; la prueba de persistencia de nuevos cambios realizados sin conexión queda por confirmar.